

SQL

"Good Queries Make Better Developers"
😊

20 Queries Every Developer Should Know

From Basic SELECT → Advanced Window Functions

- ✓ Practical Examples
- ✓ Real-world Use Cases
- ✓ Interview Focused
- ✓ Handwritten Notes
- ✓ Easy to Understand

Level up your SQL skills!



Data Drives Everything

SELECT

JOIN

WHERE

GROUP BY

HAVING

ORDER BY

DISTINCT

SUBQUERY

CTE

WINDOW FUNCTIONS

"A language that lets you talk to Data!"

```
SELECT *  
FROM opportunities  
WHERE skills = 'Learning'  
AND growth = 'Infinite';
```

Same Data . Different Questions . Bigger Insights



Keep Querying
Keep Growing



Sample Database

We will use this data for all the queries in this series! 😊

We will work with a simple company database having 3 tables :

- **employees** - information about employees
- **departments** - list of departments
- **orders** - customer orders (for few advanced queries)

Same Data Used Across All Examples

- Easy to Follow
- Better Understanding

😊

1 employees → Stores employee details

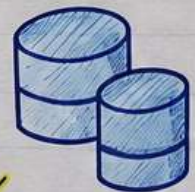
id	name	department_id	salary	manager_id
1	Abhishek	1	120000	NULL
2	Priya	2	90000	1
3	Rahul	1	80000	1
4	Sneha	3	75000	2
5	Karan	2	95000	2

- id → unique id of employee
- department_id → refers to departments.id
- manager_id → refers to employees.id (self reference)

2 departments → Stores department information

id	department_name
1	Engineering
2	Marketing
3	Human Resources
4	Finance

Each department has a unique id which is used in employees.department_id



Multiple Tables Connected using Relationships

3 orders → Stores customer order details

id	customer_id	amount	order_date
101	1	2500	2024-01-10
102	2	1800	2024-01-12
103	1	3200	2024-01-15
104	3	1250	2024-01-18
105	2	4000	2024-01-20

We will use this table for queries related to aggregations, window functions and real-world scenarios.

Key Takeaway: A real-world database usually has multiple related tables. We can combine them using JOINS to get meaningful insights! 😊

Basic Queries

Let's start with some simple but important queries!

"Good Developers Start with Good Basics" 😊

- ① **Select all records** → Retrieves all columns and all rows from a table.

```
SELECT *
FROM employees;
```

Output (first 3 rows):

id	name	department_id	salary	manager_id
1	Abhishek	1	120000	NULL
2	Priya	2	90000	1
3	Rahul	1	80000	1

💡 **Tip:** Avoid using SELECT * in production. Fetch only the required columns.

- ② **Select specific columns** → Retrieves only the columns you need.

```
SELECT name, salary
FROM employees;
```

Output:

name	salary
Abhishek	120000
Priya	90000
Rahul	80000
Sneha	75000
Karan	95000

Why?

- Reduces data transfer
- Improves performance
- Makes query output clean and readable 😊

- ③ **Use Aliases (AS)** → Gives a temporary name to a column or table.

```
SELECT name AS employee_name,
       salary AS emp_salary
FROM employees;
```

Output:

employee_name	emp_salary
Abhishek	120000
Priya	90000
Rahul	80000

Useful for:

- Readable output
- Custom column names
- Used in reports and dashboards

- ④ **Use Expressions** → You can also perform simple calculations in SELECT.

```
SELECT name,
       salary,
       salary * 12 AS annual_salary
FROM employees;
```

Output:

name	salary	annual_salary
Abhishek	120000	1440000
Priya	90000	1080000
Rahul	80000	960000

Use cases:

- Calculate derived values
- Format data
- Useful in reports 😊

Key Takeaway:

- Use only required columns.
- Aliases and expressions make your queries more flexible and readable.

Small Queries Big Impact!

Filtering & Sorting Data

"Filter Smart Query Fast"

Use WHERE and ORDER BY to get exactly what you need!

5 Filter records using WHERE → Retrieves rows that satisfy a condition.

```
SELECT *
FROM employees
WHERE salary > 80000;
```

Output :

id	name	department_id	salary	manager_id
1	Abhishek	1	120000	NULL
2	Priya	2	90000	1
5	Karan	2	95000	2

 **Tip:** You can use multiple conditions using AND, OR.

Example: SELECT * FROM employees
WHERE salary > 80000 AND department_id = 2;

6 Use Comparison Operators → Common operators used in WHERE clause.

= Equal to
!= Not equal to
> Greater than
< Less than
>= Greater than or equal to
<= Less than or equal to

Example:

```
SELECT name, salary
FROM employees
WHERE salary >= 90000;
```

You can also use multiple conditions!
WHERE department_id = 1
AND salary < 100000

7 Sort results using ORDER BY → Sorts the result set in ascending or descending order.

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Output (top 5):

name	salary
Abhishek	120000
Karan	95000
Priya	80000
Rahul	80000
Sneha	75000

ORDER BY can be:

- ASC → Ascending (default)
- DESC → Descending

You can also sort by multiple columns.

Example:

ORDER BY department_id ASC,
salary DESC;

8 Use LIMIT (or TOP) to restrict results → Returns only a specified number of rows.

```
SELECT *
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Output :

id	name	salary
1	Abhishek	120000
5	Karan	95000
2	Priya	90000

 **Note:**

In MySQL → use LIMIT
In SQL Server → use TOP
In PostgreSQL → use LIMIT

Key Takeaway: WHERE filters the data. ORDER BY sorts the data. LIMIT controls the number of rows.

Unique Values, Pattern Matching and Aggregate Functions

Small Queries
Big Insights

9 Get unique values using **DISTINCT** → Returns only distinct (unique) values.

```
SELECT DISTINCT department_id
FROM employees;
```

Output :

department_id
1
2
3



Use case :

- Find unique departments
- Unique countries
- Unique users
- Often used with COUNT()

10 Search data using **LIKE** → Search for a pattern in a column.

```
SELECT name
FROM employees
WHERE name LIKE 'A%';
```

Output :

name
Abhishek
Amit

Pattern Matching :

- 'A%' → Starts with A
- '%a' → Ends with a
- '%ha%' → Contains 'ha'
- '_a%' → Second letter is 'a'

11 Count, Sum, Avg, Min, Max → Aggregate functions perform calculations on a set of rows.

```
SELECT
COUNT(*) AS total_employees,
SUM(salary) AS total_salary,
AVG(salary) AS avg_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees;
```

Output (example) :

total_employees	total_salary	avg_salary	min_salary	max_salary
5	465000	93000	75000	120000



Notes :

- COUNT(*) → counts all rows (including NULL)
- COUNT(column) → counts only non-null values
- These functions are very useful in reports and dashboards.

12 Group data using **GROUP BY** → Groups rows with the same value.

```
SELECT department_id,
COUNT(*) AS total_employees,
AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id;
```

Output :

department_id	total_employees	avg_salary
1	2	100000
2	2	92500
3	1	75000

Use case :

- Get department-wise count
- Find average salary per dept
- Used in analytics and business reports

Key Takeaway :

- DISTINCT removes duplicates.
- LIKE helps in pattern matching.
- Aggregate functions (COUNT, SUM, AVG, MIN, MAX) give quick insights.
- GROUP BY helps in summarizing data.

Practice Queries
Build Confidence!

Joins in SQL

Combine data from multiple tables to get meaningful insights!

"Different Tables One Story" 😊

Sample Tables (from Page 2)

employees

id	name	department_id	salary
1	Abhishek	1	120000
2	Priya	2	90000
3	Rahul	1	80000
4	Sneha	3	75000
5	Karan	2	95000

departments

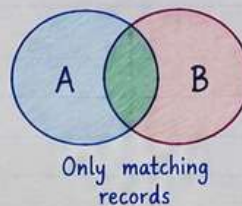
id	department_name
1	Engineering
2	Marketing
3	HR
4	Finance

Why Joins?

- Data is usually stored in multiple tables.
- Joins help us combine data using a related column (e.g. foreign key).
- Very common in real-world applications and interviews.

13 INNER JOIN → Returns only matching records from both tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id;
```

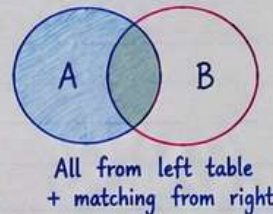


Output :

name	department_name
Abhishek	Engineering
Priya	Marketing
Rahul	Engineering
Sneha	HR
Karan	Marketing

14 LEFT JOIN → Returns all records from left table and matching records from right table.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id;
```

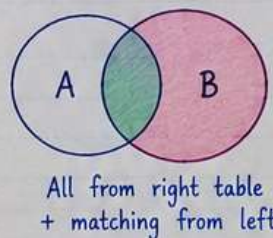


Output :

name	department_name
Abhishek	Engineering
Priya	Marketing
Rahul	Engineering
Sneha	HR
Karan	Marketing

15 RIGHT JOIN → Returns all records from right table and matching records from left table.

```
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d
ON e.department_id = d.id;
```

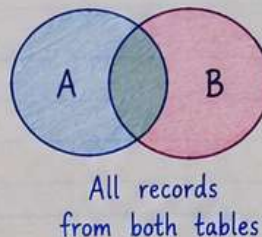


Output :

name	department_name
Abhishek	Engineering
Priya	Marketing
Rahul	Engineering
Sneha	HR
Karan	Marketing
NULL	Finance

16 FULL OUTER JOIN → Returns all records from both tables.

```
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
ON e.department_id = d.id;
```



Output :

name	department_name
Abhishek	Engineering
Priya	Marketing
Rahul	Engineering
Sneha	HR
Karan	Marketing
NULL	Finance

Key Takeaway:

- Use appropriate JOIN based on your requirement.
- Most commonly used: **INNER JOIN** and **LEFT JOIN**.
- Joins are powerful for combining related data across tables.

"Joins Connect The Dots!" 😊

GROUP BY and HAVING

Group data and apply conditions on groups !

"Data Speaks Louder in Groups" 😊

17 Count employees in each department using GROUP BY

→ Groups rows and performs aggregation.

```
SELECT department_id,
       COUNT(*) AS total_employees
FROM   employees
GROUP BY department_id;
```

Output :

department_id	total_employees
1	2
2	2
3	1



Note :

- **GROUP BY** creates groups based on the column(s)
- Aggregate functions (COUNT, SUM, AVG, etc.) are usually used with GROUP BY.

18 Find departments with average salary > 80000 using HAVING

```
SELECT department_id,
       AVG(salary) AS avg_salary
FROM   employees
GROUP BY department_id
HAVING AVG(salary) > 80000;
```

Output :

department_id	avg_salary
1	100000
2	92500



HAVING vs WHERE

- **WHERE** → filters individual rows (before grouping)
- **HAVING** → filters grouped results (after grouping) 😊

Example Difference :

WHERE (filters rows)

```
SELECT *
FROM   employees
WHERE  salary > 80000;
```

→ Returns all employees with salary > 80000

HAVING (filters groups)

```
SELECT department_id, AVG(salary)
FROM   employees
GROUP BY department_id
HAVING AVG(salary) > 80000;
```

→ Returns only departments where average salary > 80000

Remember :

- Use **WHERE** for row-level conditions.
- Use **HAVING** for group-level conditions.
- You cannot use **HAVING** without **GROUP BY**. 😊

19 Get total salary expense per department (with department name)

→ Using JOIN + GROUP BY

```
SELECT d.department_name,
       SUM(e.salary) AS total_salary
FROM   employees e
JOIN   departments d
      ON e.department_id = d.id
GROUP BY d.department_name
ORDER BY total_salary DESC;
```

Output :

department_name	total_salary
Engineering	200000
Marketing	185000
Finance	95000
HR	75000

Use cases :

- Find total/average per category (department, product, user, etc.)
- Used in reports, dashboards and analytics. 😊

Key Takeaway:

- **GROUP BY** helps in summarizing data.
- **HAVING** is used to filter grouped data.
- Combine with **JOIN** to get meaningful insights from multiple tables.

Group Analyze
Make Better Decisions! 😊

More Joins & Set Operations

Explore different types of joins and ways to combine results!

"Different Perspectives Reveal Bigger Data"

(20) SELF JOIN → Join a table with itself (useful for hierarchical data).

```
SELECT e.name AS employee,
       m.name AS manager
FROM   employees e
LEFT JOIN employees m
      ON e.manager_id = m.id;
```

Output :

employee	manager
Abhishek	NULL
Priya	Abhishek
Rahul	Abhishek
Sneha	Priya
Karan	Priya

Use case :

- Find reporting structure
- Employee → Manager
- Hierarchical data



Tip: Use **LEFT JOIN** to include top-level employees (with no manager).

(21) CROSS JOIN → Returns the Cartesian product (all possible combinations).

```
SELECT e.name, d.department_name
FROM   employees e
CROSS JOIN departments d;
```

Output (sample):

name	department_name
Abhishek	Engineering
Abhishek	Marketing
Priya	Engineering
Priya	Marketing
...	...

Use case :

- Generate all possible combinations
- Useful in reporting, testing and data generation



Note:

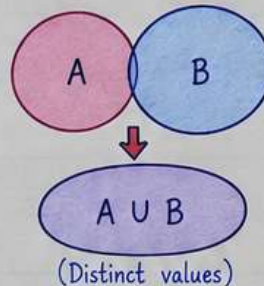
Be careful! Can return a very large result set.

(22) UNION → Combines the result sets of two queries (removes duplicates).

```
SELECT name FROM employees
UNION
SELECT name FROM temp_employees;
```

Output :

name
Abhishek
Priya
Rahul
Sneha
Karan



Use case :

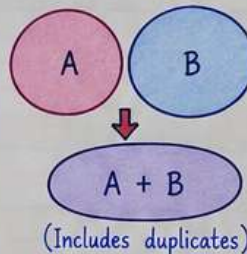
- Combine similar data from multiple tables
- Removes duplicate rows

(23) UNION ALL → Combines the result sets and keeps duplicates.

```
SELECT name FROM employees
UNION ALL
SELECT name FROM temp_employees;
```

Output :

name
Abhishek
Priya
Abhishek
Rahul
...



Use case :

- Combine data without distinct filtering
- Faster than **UNION**
- Used in logs, history data, and analytics

Key Takeaways:

- Use the right **JOIN** based on your requirement.
- **INNER JOIN** → matching records, **LEFT/RIGHT JOIN** → include non-matching.
- **UNION** → removes duplicates, **UNION ALL** → keeps duplicates.
- **SELF JOIN** is useful for hierarchical data.
- **CROSS JOIN** gives all possible combinations (use carefully).

Joins Expand Possibilities!

Subqueries and CTEs

Write modular and powerful queries using subqueries and CTEs!

"Break Problems into Smaller Queries" 😊

24 Subquery (Nested Query) → A query inside another query.

```
SELECT name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

What it does ?

- Inner query runs first (finds avg salary)
- Outer query uses the result to filter rows

Output (example):

name	salary
Abhishek	120000
Karan	95000

Use cases:

- Filtering with aggregated values
- Checking existence (WHERE ... IN/EXISTS)
- Comparing with derived results

25 Types of Subqueries → Subqueries can be of different types.

1. Scalar Subquery → Returns single value
2. Single Row Subquery → Returns single row
3. Multiple Row Subquery → Returns multiple rows
4. Correlated Subquery → Depends on outer query

Example (Correlated):

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```

Correlated subquery runs for each row of the outer query! 😊

26 EXISTS and IN → Used to check for the existence of records.

Using IN:

```
SELECT name
FROM employees
WHERE department_id IN (
    SELECT id FROM departments
    WHERE department_name = 'Engineering'
);
```

Using EXISTS:

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM departments d
    WHERE d.id = e.department_id
    AND d.department_name = 'Engineering'
);
```



Tip:

- IN → Good for small result sets.
- EXISTS → Better for large datasets (more efficient).
- EXISTS stops at first match, IN checks all values.

27 Common Table Expression (CTE) → A temporary result set that you can reference in a query using WITH.

```
WITH high_paid AS (
    SELECT name, salary
    FROM employees
    WHERE salary > 80000
)
SELECT * FROM high_paid;
```

Output (example):

name	salary
Abhishek	120000
Priya	90000
Karan	95000

Use cases:

- Improve readability
- Break complex queries into smaller parts
- Reuse the same result set multiple times
- Useful in analytical queries

Key Takeaways:

- Subqueries help in filtering and deriving results.
- EXISTS is often more efficient than IN.
- CTEs make complex queries simple and readable.
- Choose the right approach based on your use case and data size.

Simpler Queries Stronger Insights! 😊

Indexes and Query Optimization

Make your queries faster and your applications scalable!

"Faster
Queries
Happier
Users"

28 What is an Index?

→ Improves search performance by creating a sorted structure (like a book index).

- Indexes help the database find rows quickly without scanning the entire table.
- They are stored separately from the data.
- Speeds up **SELECT** queries (especially with **WHERE**, **JOIN**, **ORDER BY**).
- Comes with a trade-off: extra storage and slower write operations (**INSERT/UPDATE/DELETE**).

Example:

```
CREATE INDEX idx_salary
ON employees(salary);
```

This creates an index on the salary column for faster searches.



Analogy:

- Just like a book's index helps you find a topic quickly, a database index helps you find rows without scanning every record.

29 Query with and without Index

Without Index (Full Table Scan)

```
SELECT * FROM employees
WHERE salary = 100000;
```

→ Database scans every row.
(Slower for large tables)

With Index (Index Scan)

```
SELECT * FROM employees
WHERE salary = 100000;
```

→ Uses the index to find matching rows directly.
(Much faster)

Execution Difference (Example)

Query Type	Rows Scanned	Time (approx)
Without Index	1,000,000	1200 ms
With Index	10	5 ms



Note:

- Actual numbers depend on data size, database and hardware.
- Indexes are most useful on large tables.

30 Best Practices for Better Performance

1. Use indexes on frequently searched columns (**WHERE**, **JOIN**, **ORDER BY**).
2. Avoid **SELECT *** → fetch only required columns.
3. Use proper data types (**INT** instead of **VARCHAR** for IDs).
4. Keep statistics updated (helps query optimizer).
5. Avoid functions on indexed columns (e.g., **WHERE YEAR(date) = 2023** can be slow).
6. Use **LIMIT** for large results.
7. Analyze slow queries using **EXPLAIN**.

Example (Avoid vs Better):

❌ Avoid

```
SELECT *
FROM employees
WHERE YEAR(join_date) = 2023;
```

This can't use the index efficiently (because of the function **YEAR()**).

✅ Better

```
SELECT *
FROM employees
WHERE join_date > '2023-01-01'
AND join_date < '2024-01-01';
```

This can use the index on **join_date**.



Tip: Use **EXPLAIN** to see how your query is executed.

```
EXPLAIN SELECT * FROM employees
WHERE salary = 100000;
```

→ Shows whether index is used or a full table scan is happening.

Key
Takeaways:

- Indexes speed up read operations.
- Use the right index on the right column.
- Write queries that can leverage indexes.
- Always analyze and optimize slow queries.
- Good query design = Better performance = Happier users!

Optimize
Today
Scale Tomorrow!

Real-World Examples and Use Cases

Let's solve some practical problems using what we learned!

"Queries Solve Real Problems!"

31 Find Top 3 Highest Paid Employees in Each Department

```
SELECT department_name, name, salary
FROM (
  SELECT e.name, e.salary, d.department_name,
         ROW_NUMBER() OVER (
           PARTITION BY d.department_id
           ORDER BY e.salary DESC
         ) as rn
  FROM employees e
  JOIN departments d ON e.department_id = d.department_id
) t
WHERE rn <= 3
ORDER BY department_name, salary DESC;
```

Output (sample):

department_name	name	salary
Engineering	Abhishek	120000
Engineering	Priya	110000
Engineering	Karan	105000
Marketing	Sneha	95000
Marketing	Rahul	90000
Marketing	Neha	85000

Use case:

- Find top performers in each department.
- Useful in HR reports and analytics.
- Uses window functions (ROW_NUMBER).

Window functions make advanced analysis easy!

32 Find Departments with No Employees

→ Use LEFT JOIN with NULL check.

```
SELECT d.department_id, d.department_name
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
WHERE e.employee_id IS NULL;
```

Output (sample):

department_id	department_name
4	Finance
5	Legal

Use case:

- Find unused or inactive departments.
- Useful for data cleanup and organization.

33 Count Employees in Salary Ranges

→ Use CASE inside aggregate function.

```
SELECT
  SUM(CASE WHEN salary < 60000 THEN 1 ELSE 0 END) AS low_salary,
  SUM(CASE WHEN salary BETWEEN 60000 AND 100000 THEN 1 ELSE 0 END) AS mid_salary,
  SUM(CASE WHEN salary > 100000 THEN 1 ELSE 0 END) AS high_salary
FROM employees;
```

Output (sample):

low_salary	mid_salary	high_salary
2	3	2

Use case:

- Categorize data into ranges.
- Useful for reporting and dashboards.

34 Find Duplicate Salaries

→ Use GROUP BY with HAVING.

```
SELECT salary, COUNT(*) AS count
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

Output (sample):

salary	count
75000	2
90000	2

Use case:

- Find duplicate or suspicious data.
- Useful for data quality checks.

Helps in maintaining clean and reliable data!

35 Combine Multiple Insights

→ A real-world example using multiple concepts together.

```
SELECT d.department_name,
  COUNT(e.employee_id) as total_employees,
  AVG(e.salary) as avg_salary,
  MAX(e.salary) as max_salary,
  MIN(e.salary) as min_salary
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY total_employees DESC;
```

Output (sample):

department_name	total_employees	avg_salary	max_salary	min_salary
Engineering	3	105000	120000	90000
Marketing	2	87500	95000	80000
HR	1	75000	75000	75000
Finance	0	NULL	NULL	NULL

- Use real-world problems to strengthen your SQL skills.
- Combine multiple concepts (JOIN, GROUP BY, HAVING, CTE, etc.).
- Always think about the use case and data you are working with.
- Practice with different datasets to build confidence.

Practice Builds Better Developers!

Key Takeaways:

Common Mistakes and Interview Questions

Learn from common mistakes and be ready for interviews!

"Small Mistakes Teach Big Lessons"

36 Common Mistakes while using GROUP BY

✗ Mistakes :

- Selecting non-aggregated columns that are not in **GROUP BY**.
- Using **WHERE** instead of **HAVING** for aggregate conditions.
- Forgetting that **NULL** values are grouped together.
- Not using proper aliases for readability.
- Using **GROUP BY** when a window function is more appropriate.

Wrong vs Correct :

✗ Wrong :

```
SELECT department_id, name, COUNT(*)
FROM employees
GROUP BY department_id; → name is not aggregated or grouped.
```

✓ Correct :

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

Key Takeaway :

- Every selected column must be either aggregated or part of **GROUP BY**.
- Use **HAVING** for conditions on aggregated data.
- Always check for **NULL** values.

37 Common Mistakes while using JOIN

✗ Mistakes :

- Using **INNER JOIN** when **LEFT JOIN** is required (missing data).
- Incorrect join condition (leads to wrong results).
- Not understanding many-to-many relationships (duplicate rows).
- Forgetting to select the right columns (can create ambiguity).
- Using **CROSS JOIN** accidentally (huge result sets).

Wrong vs Correct :

✗ Wrong :

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id; → This will not show employees without a department.
```

✓ Correct :

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

Key Takeaway :

- Choose the right join based on the requirement.
- Always verify the join condition.
- Be aware of duplicate rows.
- Use table aliases to avoid ambiguity.

38 Interview Questions (with answers)

Q1. What is the difference between **WHERE** and **HAVING**?

Ans:

- **WHERE** filters individual rows (before grouping).
- **HAVING** filters grouped results (after aggregation).

Q2. Can we use aliases in **GROUP BY**?

Ans: Yes, in most SQL databases you can use column aliases in **GROUP BY** (e.g. dept AS d → **GROUP BY** d). But it's safer to use the actual column name for better compatibility.

Q3. What is the purpose of **HAVING**?

Ans: **HAVING** is used to filter groups based on aggregate functions like **COUNT**, **SUM**, **AVG**, etc.

Q4. What is a **CROSS JOIN**?

Ans: **CROSS JOIN** returns the Cartesian product of two tables (all possible combinations of rows). Be careful, as it can produce a very large result set.

Q5. How do you find duplicate records in a table?

Ans: Use **GROUP BY** with **HAVING COUNT(*) > 1** on the relevant columns.

Q6. What is a **CTE** and when to use it?

Ans: A **CTE** (Common Table Expression) is a temporary result set that can be referenced within a query using **WITH**. It improves readability and helps in writing modular queries.

Final Takeaways:

- Practice writing queries with real-world datasets.
- Understand common mistakes to avoid errors in production.
- Be ready to explain your queries in interviews.
- Think about performance and scalability.
- SQL is not just about writing queries, but solving real business problems!

Learn Practice Apply Grow!

Advanced Topics and Next Steps

Take your SQL skills to the next level!

"Keep Learning
Keep Building"
😊

39 Window Functions → Perform calculations across a set of rows related to the current row.

```
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department
                   ORDER BY salary DESC) as dept_rank
FROM employees;
```

Output (sample):

name	department	salary	dept_rank
Abhishek	Engineering	120000	1
Priya	Engineering	110000	2
Rahul	Engineering	105000	3
Sneha	Marketing	95000	1

Use cases:

- Rankings (top N)
- Running totals
- Moving averages
- Percentiles
- Comparing rows within a group

40 Subqueries vs CTEs vs Temporary Tables → Different ways to break complex queries.

Approach	Description	Pros	Cons	When to use
Subquery	Query inside another query	Simple and quick	Can be hard to read (especially nested)	Small and simple tasks
CTE (WITH)	Temporary result set using WITH	More readable, modular	Not always materialized (performance depends)	Complex queries, reusability
Temporary Table	Stores result set temporarily	Can be indexed, reused multiple times	Needs extra steps (create & drop)	Large datasets, multiple operations

CTEs make queries modular and easier to read!
😊

41 Performance Tuning Tips → Write efficient queries and make the most of your database.

- Use proper indexes (on frequently used columns).
- Avoid `SELECT *` (fetch only required columns).
- Use `EXPLAIN` to analyze query execution.
- Filter data early (in `WHERE` clause).
- Choose the right `JOIN` type.
- Avoid functions on indexed columns.
- Keep statistics updated.
- Denormalize only when necessary.
- Use appropriate data types.

Example: Using `EXPLAIN`

```
EXPLAIN SELECT name, salary
FROM employees
WHERE department = 'Engineering'
AND salary > 100000;
```

Key Takeaway:

- Always analyze before optimizing.
- Small changes can bring big improvements.
- Good query design saves time and cost.

42 Next Steps in SQL Learning → Continue exploring and build real-world projects.

- Practice on real datasets (e.g., Kaggle, LeetCode, HackerRank).
- Try solving business problems (sales analysis, user analytics, etc.).
- Learn database-specific features (PostgreSQL, MySQL, SQL Server).
- Explore advanced topics (stored procedures, triggers, views).
- Work on a personal project (e.g., E-commerce database).
- Read query execution plans and optimize them.
- Combine SQL with other skills (Python, Data Visualization).

Suggested Practice Platforms:

- LeetCode (Database)
- HackerRank (SQL)
- StrataScratch (Real interview questions)
- Kaggle (Real datasets)
- Mode Analytics (SQL tutorials)

Final Note:

SQL is a powerful skill that goes beyond writing queries. The more you practice, explore real-world problems, and optimize your queries, the more valuable you become as a developer or data professional!

Better Queries
Brighter Future!
😊

SQL Practice Problems

Let's solve some practice questions step by step!

"Practice More Build Confidence"

43 Find the Second Highest Salary → Common interview question!

Problem :

Find the second highest salary from the employees table.

Table : employees

id	name	salary
1	Abhishek	120000
2	Priya	110000
3	Karan	105000
4	Sneha	95000

Query :

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
    SELECT MAX(salary)
    FROM employees
);
```

Output :

second_highest
110000

Key Takeaways :

- Use a subquery to find the highest salary first.
- Then fetch the maximum salary less than that.
- Works for distinct values.
- Handle NULL values if required.

44 Find Employees with Salary Above Department Average → Slightly advanced!

Problem :

Find all employees whose salary is greater than the average salary of their department.

Table : employees

id	name	department	salary
1	Abhishek	Engineering	120000
2	Priya	Engineering	90000
3	Rahul	Marketing	95000
4	Neha	Marketing	105000

Query :

```
SELECT e.id, e.name, e.department, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department
);
```

Output :

id	name	department	salary
1	Abhishek	Engineering	120000
4	Neha	Marketing	105000

Key Takeaways :

- Use a correlated subquery to compare with department average.
- Helps in real-world analysis (top performers).
- Can also be solved using JOIN with aggregated table.

Compare Analyze Find Insights!

45 Delete Duplicate Records → Useful in data cleanup!

Problem :

Delete duplicate records keeping the latest one (highest id) based on email.

Table : users

id	name	email	created_at
1	Abhishek	a@example.com	2024-01-01
2	Abhishek	a@example.com	2024-02-01
3	Priya	p@example.com	2024-01-10
4	Priya	p@example.com	2024-03-01

Query :

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MAX(id)
    FROM users
    GROUP BY email
);
```

Output :

id	name	email	created_at
2	Abhishek	a@example.com	2024-02-01
4	Priya	p@example.com	2024-03-01

Key Takeaways :

- Use GROUP BY with MAX(id) to keep latest record.
- Helps in removing duplicate data.
- Be careful before deleting data in production.
- You can also use ROW_NUMBER() with CTE for more control.

Final Takeaways:

- Practice diverse SQL problems (basic to advanced).
- Understand different approaches to solve the same problem.
- Write clean, readable and optimized queries.
- Real-world practice will make you confident for interviews and projects!

Same Concepts Bigger Opportunities!

Revision Cheat Sheet and Final Tips

Quick recap, key concepts and tips to master SQL!

"Revise
Practice
Apply
Succeed"

46 SQL Quick Revision (Key Concepts)

1. **SELECT** - Fetch data from a table.
2. **WHERE** - Filter rows based on condition.
3. **DISTINCT** - Remove duplicate rows.
4. **ORDER BY** - Sort the result set.
5. **GROUP BY** - Group rows for aggregation.
6. **HAVING** - Filter groups after aggregation.
7. **JOIN** - Combine data from multiple tables.
8. **SUBQUERY** - Query inside another query.
9. **CTE (WITH)** - Temporary result set.
10. **WINDOW FUNCTIONS** - Perform calculations across rows.
11. **INDEX** - Improves query performance.
12. **EXPLAIN** - Analyze query execution plan.
13. **DML** - **INSERT**, **UPDATE**, **DELETE**.
14. **DDL** - **CREATE**, **ALTER**, **DROP**.
15. **DCL** - **GRANT**, **REVOKE** (access control).

Common Interview Question Types

1. Find highest / lowest / Nth highest value.
2. Find duplicates and delete them.
3. Find records based on conditions (**WHERE**, **HAVING**).
4. Use of **JOINS** (**INNER**, **LEFT**, **RIGHT**, **FULL**).
5. Grouping and aggregation problems.
6. Subqueries vs CTEs.
7. Window function problems (**RANK**, **ROW_NUMBER**, **LAG**, **LEAD**).
8. Real-world scenarios (e.g., top sellers, department-wise stats).
9. Query optimization (index usage, **EXPLAIN**).
10. Data cleanup and transformation.

Pro Tips for Interviews

- ✓ Write clean and readable queries.
- ✓ Think about edge cases (NULL values, duplicates).
- ✓ Use meaningful aliases.
- ✓ Explain your thought process.
- ✓ Know the time and space complexity of your query.
- ✓ Practice on real datasets.
- ✓ Be ready to optimize queries (indexes, query plan).
- ✓ Try solving the same problem in multiple ways.
- ✓ Communicate clearly and confidently.

It's not
just about
writing SQL,
it's about
thinking with
SQL!

47 Suggested Practice Roadmap

- Stage 1: Basics (**SELECT**, **WHERE**, **ORDER BY**)
- Stage 2: Aggregations (**GROUP BY**, **HAVING**)
- Stage 3: Joins (**INNER**, **LEFT**, **RIGHT**, **FULL**)
- Stage 4: Subqueries and CTEs
- Stage 5: Window Functions
- Stage 6: Real-world Problems (mix of all concepts)
- Stage 7: Optimization and **EXPLAIN**.
- Stage 8: Timed Practice (simulate interview)

Recommended Practice Platforms

- LeetCode (Database problems)
- HackerRank (SQL)
- StrataScratch (Real interview questions)
- Kaggle (Real datasets)
- Mode Analytics (SQL tutorials)
- DataLemur (Interview focused)

Consistent
Practice
Creates
Results!

48 Final Takeaways

- SQL is a must-have skill for every data-driven role (SDE, Data Analyst, Data Engineer, etc.).
- Focus on understanding concepts, not just memorizing queries.
- Practice regularly with real-world datasets.
- Solve a mix of easy, medium and hard problems.
- Learn query optimization techniques.
- Use SQL in personal projects to build confidence.
- Keep revising and take mock tests before interviews.
- The more you practice, the better and faster you will get!

"A query
a day
keeps the
interview fear
away!"

Learn
Practice
Build
Grow!